

INTEGRATION DOCUMENT

INDEX

SR NO.	CONTENT	PAGE NO.
1	Introduction	1
2	Environments	1
3	Encryption Decryption Logic	2
4	Process Flow & Integration Methods	3
A	Aggregator Hosted (Non-Seamless)	3
B	API Integration	13
5	Encryption API	24
6	Decryption API	25

1. Introduction:

This document describes the different methods of Integration between Merchant and **yagoutpay**.

Yagout services provides trustworthy and feasible solutions for the registered merchants. Register as a merchant with us to know more about the features of **yagout** Payment Gateway services.

Yagout Payment Integration kit allows merchants to instantly collect payments from their users using various payment modes like Credit cards, Debit cards & Wallets.

2. Environments:

We have our Testing and Production Environment, at initial stage merchants will be using Test credentials and post successful integration the merchant will be provided with LIVE merchant credentials for the Production environment.

In the Test environment we have a Simulator flow designed to test Success/Failure cases to ease merchants complete the integration.

- Our **T**est and **P**roduction environments are separate.

Yagoutpay Test Environments:

Post URL	https://uatcheckout.yagoutpay.com/ms-transaction-core-1-0/paymentRedirection/checksumGatewayPage
Merchant ID	**Shared by us**
Encryption Key	**Shared by us**
Aggregator ID	**yagout**

Yagoutpay Production Environments:

Post URL	**Shared by us**
Merchant ID	**Shared by us**
Encryption Key	**Shared by us**
Aggregator ID	**yagout**

3. Encryption Decryption Logic.

ALL the parameters are encrypted and decrypted using Aes 256 logic shared below with the help of merchant encryption key.

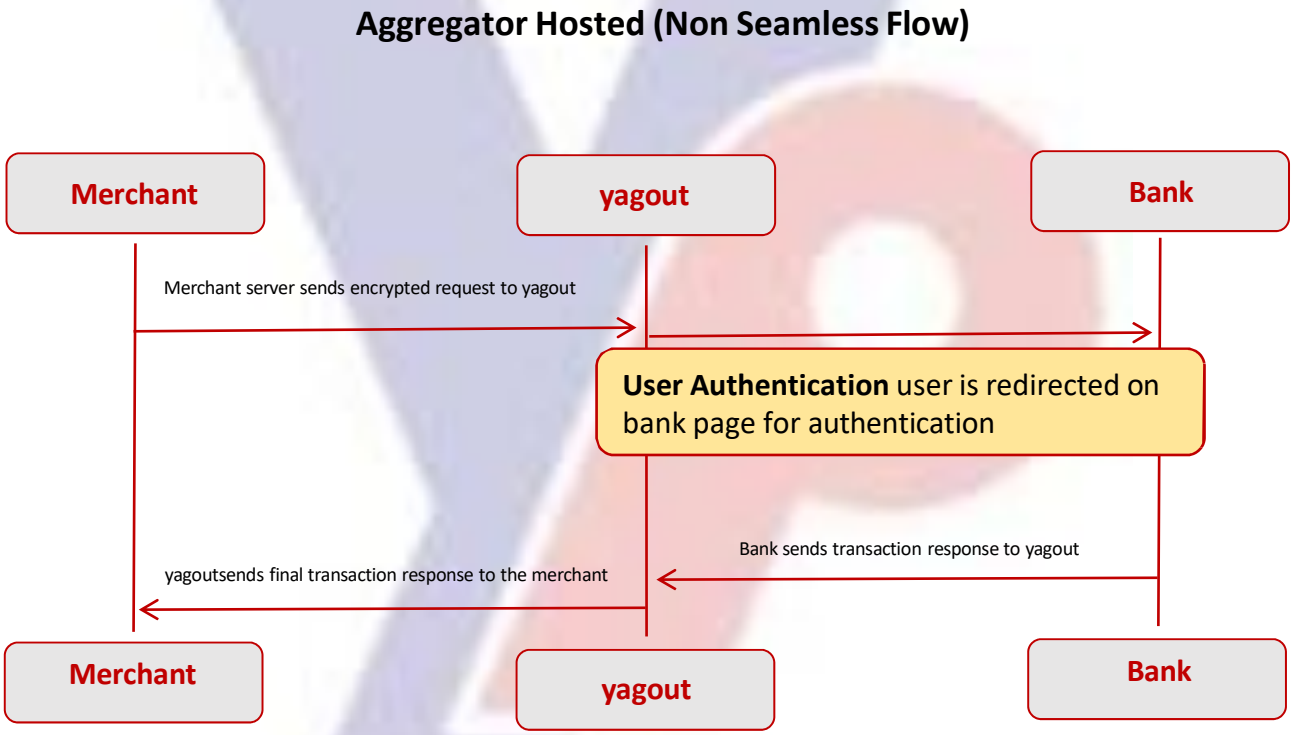
```
function encrypt($text, $key, $type)
{
$iv = "0123456789abcdef";
$size = 16;
$pad = $size - (strlen($text) % $size);
$padtext = $text . str_repeat(chr($pad), $pad);
$crypt = openssl_encrypt($padtext, "AES-256-CBC",
                base64_decode($key), OPENSSL_RAW_DATA
                | OPENSSL_ZERO_PADDING, $iv);
return base64_encode($crypt);
}
function decrypt($crypt, $key, $type)
{
$iv = "0123456789abcdef";
$crypt = base64_decode($crypt);
$padtext = openssl_decrypt($crypt, "AES-256-CBC",
                base64_decode($key), OPENSSL_RAW_DATA
                | OPENSSL_ZERO_PADDING, $iv);
$pad = ord($padtext[strlen($padtext) - 1]);
if ($pad > strlen($padtext)) {
return false;
}
if (strspn($padtext, $padtext[strlen($padtext) - 1], strlen($padtext) -
$pad) != $pad) {
  $text = "Error";
}
$text = substr($padtext, 0, -1 * $pad);
  return $text;
}
```

4. Process Flow & Integration Methods.

Yagoutpay supports collecting payment information using Aggregator Hosted method. This method is designed to support a smooth user-experience.

A. Aggregator Hosted (Non-Seamless):

Avoid struggling to develop and manage your own checkout page.Use the customized payment page provided by **yagoutpay** which enables you to collect payment from the customer.



Process flow:

1. Customer selects product/service on your website and proceeds to make payment.
2. Customer is redirected to the yagout payment page where payment information is entered by the customer.
3. On submission of the transaction information, yagout initiates the authorization process by connecting to the relevant bank / processing organization.
4. On receiving the authorization status from the bank, yagout sends the response back to your website with the transaction status.

YagoutPay Test Environments for Aggregator Hosted (Non-Seamless)

Post URL	https://uatcheckout.yagoutpay.com/ms-transaction-core-1-0/paymentRedirection/checksumGatewayPage
Merchant ID	202505060003
Encryption Key	SjgTPThS1TWrD06E1FjRC3BoIg0XEh/xJ2IC+k+tVVM=
Aggregator ID	yagout

Request Parameters:

Section: Txn_Details (Required Parameters)			
Parameter Name	**Mandate**	**Type (length)**	**Description**
ag_id	YES	Alphabets (11)	Aggregator id is "yagout".
me_id	YES	Numeric(20)	Merchant ID issued to you by yagout.
order_no	YES	Alphanumeric (70)	This ID is used by merchants to identify the order. Ensure that you send a **unique id** with each request. As it generates a unique payment reference number for each order which is sent by the merchant.
Amount	YES	Numeric(10,2)	This is your Transaction Amount. You can use upto 2 decimal places. E.g.,"150.00"
Country	YES	Alphabets (3)	Country code needs to be passed, Country code as "ETH" for Ethiopia.

Currency	YES	Alphabets (3)	Currency in which you want to process the transaction. ETB - Ethiopian birr
txn_type	YES	Alphabets (10)	Use Txn_Type as "**SALE**" which is static with each request
success_url	YES	Alphanumeric (200) **Characters allowed:** Alphabet (A-Z), (a-z), Numbers, / (slash), _ (underscore)	This will be the return URL where response is accepted for successful transaction.
failure_url	YES	Alphanumeric (200 **Characters allowed** Alphabet (A-Z), (a-z Numbers, / (slash), _ (underscore)	This will be the return URL where response is accepted for failed transaction.
Channel	YES	Alphabet (10)	Use "**WEB**" in case of website-based payment or"**MOBILE**" in case of mobile app. Static value with each request.
Section: pg_details			
pg_id	No	Numeric(11)	Value must beblank for Aggregator Hosted (Non-Seamless)
Paymode	No	Alphabets	Value must beblank for Aggregator Hosted (Non-Seamless)inthe request.
Scheme	No	Numeric(11)	Value must be blank for Aggregator Hosted (Non-Seamless)
wallet_type	No	Alphabets	Enter the wallet e.g.: telebirr , mpesa, cbebirr, kacha
Section: card_details			
card_no	No	Numeric(19)	Value must beblank for Aggregator Hosted (Non-Seamless)
exp_month	No	Numeric (2)	Value must beblank for Aggregator Hosted (Non-Seamless)
exp_year	No	Numeric (4)	Value must be blank for Aggregator Hosted (Non-Seamless)

cvv	No	Numeric (3)	Value must beblank for Aggregator Hosted (Non-Seamless)

card_name	No	Alphabets (50)	Value must be blank for Aggregator Hosted (Non-Seamless)
Section: cust_details			
cust_name	Optional	Alphabets (50)	Customer Name. This is required for fraud protection.
email_id (**Mandatory**)	Yes	Alphanumeric (100) **Characters allowed:** Alphabet (A-Z), (a-z). Numbers @ (at), dot, _ (underscore)	Customer's Email ID. This is required required for fraud protection.
mobile_no (**Mandatory**)	Yes	Numeric(15)	Customer's Mobile Number. This is required required for fraud protection.
unique_id	No	Alphanumeric(100)	This is required incase of **Stored card feature**.
is_logged_in	Yes	Alphabet (1)	Pass"Y" if user is logged in.
Section: Bill_details			
bill_addres	Optional	Alphanumeric (400) **Characters allowed:** Alphabet (A-Z), (a-z). Numbers # (hash), Comma, circular brackets, /(slash), dot,- (hyphen) Space in between words	Customer's Billing Address. This is required required for fraud protection.
bill_city	Optional	Alphabets (50)	Customer's Billing City. This is required for fraud protection.
bill_state	Optional	Alphabets (50)	Customer's Billing State. This is required for fraud protection.
bill_country	Optional	Alphabets (50)	Customer's Billing Country.This is required for fraud protection.
bill_zip	Optional	Alphanumeric (20) **Characters allowed:** Alphabet (A-Z),(a-z). Numbers	Customer's Billing Zip/Pin Code.This is required for fraud protection.
Section: Ship_details			

ship_address	Optional	Alphanumeric (400) **Characters allowed:** Alphabet (A-Z), (a-z). Numbers # (hash), Comma, circular brackets, /(slash), dot,- (hyphen) Space in between words	Customer's Shipping Address. This is required in case of physical delivery of goods.
ship_city	Optional	Alphabets (50)	Customer's Shipping City. This is required in case of physical delivery of goods.
ship_state	Optional	Alphabets (50)	Customer's Shipping State. This is required in case of physical delivery of goods.
ship_country	Optional	Alphabets (50)	Customer's Shipping Country. This is required in case of physical delivery of goods.
ship_zip	Optional	Alphanumeric (20) **Characters allowed:** Alphabet (A-Z),(a-z). Numbers	Customer's Shipping Zip/Pin Code. This is required in case of physical delivery of goods.
ship_days	Optional	Numeric(11)	Approximate number of days required for delivery of goods. This is required in case of physical delivery of goods.
address_count	Optional	Numeric(11)	If customer has more than one addresses stored in your system.
Section: Item_details			
item_count	Optional	Alphanumeric (11)	Count of items purchased by customer.
item_value	Optional	Alphanumeric(200,2)	Commaseparateditem wise cost
item_category	Optional	Alphanumeric(200)	Commaseparateditem category. E.g., "DVD"
Section: UPI_details			
Upi_Id	Optional	Alphanumeric (100) **Characters allowed:** Alphabet (A-Z), (a-z). Numbers # (hash), Comma, circular brackets,/(slash), dot, -(hyphen) Space in betweenwords	Currently not required.
Section: Other_details			

udf_1	Optional	Alphanumeric (100) **Characters allowed:** Alphabet (A-Z), (a-z). Numbers # (hash), Comma, circular brackets,/(slash), dot, -(hyphen) Space in betweenwords	This parameter can be used for sending additional information about the transaction. PG will send this parameter in the response.
udf_2	Optional	Alphanumeric (100) **Characters allowed:** Alphabet (A-Z), (a-z). Numbers # (hash), Comma, circular brackets,/(slash), dot, -(hyphen) Space in betweenwords	This parameter can be used for sending additional information about the transaction. PG will send this parameter in the response.
udf_3	Optional	Alphanumeric (100) **Characters allowed:** Alphabet (A-Z), (a-z). Numbers # (hash), Comma, circular brackets,/(slash), dot, -(hyphen) Space in betweenwords	This parameter can be used for sending additional information about the transaction. PG will send this parameter in the response.
udf_4	Optional	Alphanumeric (100) **Characters allowed:** Alphabet (A-Z), (a-z). Numbers # (hash), Comma, circular brackets,/(slash), dot, -(hyphen) Space in betweenwords	This parameter can be used for sending additional information about the transaction. PG will send this parameter in the response.
udf_5	Optional	Alphanumeric (100) **Characters allowed:** Alphabet (A-Z), (a-z). Numbers # (hash), Comma, circular brackets,/(slash), dot, -(hyphen) Space in betweenwords	This parameter can be used for sending additional information about the transaction. PG will send this parameter in the response.

Note:

Value in each section will be encrypted value generated in the following way. Onlyme_id (Merchant ID) will be in plain text.

Each attribute in everysection is combined and separated with pipe (|). Make sure that you are sending the request in the **HTTP Form post method.**

Pleasefindtheformat sharedbelow:

Thetransaction request is not compatiblewiththe Postman Platform.

Transactionsareonlypermitted throughregistered merchantdomains. If "localhost" or local IP is not permitted, you will get an"**Invalid Referral URL**" error in such case.

Transaction requests must be in the manner below when it trigger to our Payment Gateway URL. Merchants must send the following parameters to the **yagout** PG, which is mandatory, but values might be blank if they are optional or not required.

Exampleof the Transactionrequest in HTTPForm post method: Sample HTMLtransaction posting:

- Make surethat you are sending the request in the **HTTP Formpost method**.
- Transactionsareonlypermittedthroughregistered merchantdomains. If "localhost" or local IP is notpermitted, you willget "**Invalid Referral URL**" error.
- Transactionrequest must be in themannerbelow when it triggers to our Payment Gateway URL.
- Merchants must send thefollowingparameters to the **yagout** PG which is mandatory, but values might be blank in case of optional or not required.

Please note only HTML Form Submission with the 'POST' method is supported.
Sending payment requests using any REST API call is not supported.

```
<! DOCTYP E html>
<htm l>
<body>
<form  name="paymentForm"  method="POST"  enctype= "application/ x-www-for-urlencod ed"

action="https://uatcheckout.yagoutpay.com/ms-transaction-core-1-0/paymentRedirection/checksumGatewayPage"  >

<input  name= " me_id"  value="  202505060003"  type="hidden">

<input  name="merchant_request"  value=
"VueAaFgKrAMO1sA+p8YKdC0r5vldTBmRzJr/tmLSx+5Kmw6lup47Q3wm2z3kKZJdbHmO3i+5ww0nzcqo1
vRIiKNqCJCPrk/nVLMnf0ehrUexFICmhuGL+qzsL24Rv3Ugs2rZHLlrRDCUeSqeHiE87zeqxWlvGJEAqKE34
DfKT7cjeNztZEDpzsfVQn0As22obQzrxVMOFulnd3LKB07JBfJmOZKB+87Yg72iYY11KA/QPWwTJ3RqXCJ
xBaogz8Lm"  type="hidden" >

<input  name= "hash "  value  =
"0FHhRpcLlgr8Hb88AaM+5uhGP3HwMueYTIzCKM5hQTW9JWbDwA0/X+k1/a46dYFw3Vw29id/b2dvp9
Y3iuzoKFGPEIlGw8U2ObdDSzef+tM="  type="hidden">

<input  type="submit"name="submit"  value="Pay  Now">
</form>
</body>
</html>
```

Note :

The hash is created using below parameters with **sha256** algorithm.

All the values are separated using tilde (~).

hash : merchant_id ~ order_no ~ amount ~ country ~ currency

Sample Request Before Encryption:

me_id: 202505060003

merchant_request:

yagout|202505060003|49340|1|ETH|ETB|SALE|http://localhost/YagoutPay/transaction//Response.php|http://localhost/YagoutPay/transaction//Response.php|WEB~|||~||||~||||Y~||||~||||||~||~~||||

hash:

LuzwiRKvTGhRXZ/Z6FX9SDKzZpEK2P5UWD1nUVfgStT/cBoxond5C6JdAQm3rPBJWUpPMKWw1kIZ9d6gfgLLAibWJtxJMPbqvUTlztlP+fs=

Sample Request After Encryption:

me_id: 202505060003

merchant_request:

VueAaFgKrAMO1sA+p8YKdG3nVX8q1aX2dZ6LqPzxY3lj+3W9gmIwOXvaHWyWw4mUR+JnftInJUGOy2CCVpsJ9OMJ4kiHnqDLa889i9G2MyOalCksMDjsRFkLlLETUGEIu8muC6//u2roewGFX/XRBtL1u4xuQ9o6nf0UkjLtC3JxFPa0F9HaZc4mUtL1iFni3kKI+UHn5pQevqv8shX/IZG1kr8Ht+LmkrPgcV2pjfDfkeZyCR3lAuXwmU3vhNdW

hash:

LuzwiRKvTGhRXZ/Z6FX9SDKzZpEK2P5UWD1nUVfgStT/cBoxond5C6JdAQm3rPBJWUpPMKWw1kIZ9d6gfgLLAibWJtxJMPbqvUTlztlP+fs=

Merchant_request will get encrypt by aes256 (Details shared below):

```
$all_values = $txn_details.'~'.$pg_details.'~'.$card_details.'~'.$cust_details.'~'.$bill_details.'~'.$ship_details. '~'.$item_details. '~'.$upi_details'~'.$other_details;
$merchant_request['merchant_request'] = $aes->encrypt($all_values, $merchant_key, 256);
```

Encrypted Response:

txn_response:
VueAaFgKrAMO1sA+p8YKdNuPZMgV55LVg9ZC1iKGwlZlOLsuh6rXX4mOs4wtFOrZQU43A1lfIdSCw8/EqNofU2uNVfV3KGqUL4+gMRB1bnWX4gvm70jsQBIam7A25cTYvSLSKwcX5GEMxwS8Y2TK2pG3fU8fjKAXlyP5Bn88nB3qDMbT7jlhpE2GRMVL7WPI

me_id: 202505060003

pg_details:
h48SfPUC72rvLVgAdLAMqI6w8jiUgujB2aYUJSJbitk7pBA3OnEQ0xpUW3mY6brNQfN+ZIEHlGu2AIbcJVFKXw==

txn_details:
uKnq0Vq1PMguO1QMdEeBdC8oYlDJNwSNijC/8HKKbBrvFGHkhTOp0EfWPsXkwJnUUg311epY8AxJ3+7Nz64Un9bn6sLopX3D4JAzxH/jl+y6iHIdt6+UorcD+E+7hSXR0Kc79I6ZUo70U2OujTXpjZ5xZKzS6u1IrQ56HC9A+2CJep5h9w2WubUB2rA1/1NuTdRJ/wVtGM0TpLSvF2AicpphtsQxyF2CJAXI1sNLHSerJvIP203poPjcw1YOQJM0

other_details: wVyWLbJUUvp7oSiptxO7Yg==

fraud_details: DIAmMIVbZZ3jyJJhbaVIwA==

Decrypted Response:

txn_response:
yagout|202505060003|56212|1.00|ETH|ETB|2025-04-30|09:56:23|2058501746006825776|AG_20250430_70701b7a0bffdb44a880|Successful|0|Successful

me_id: 202505060003

pg_details:
67ee846571e740418d688c3f|YAGOUTPAY_MIDDLEWARE|WA|7

txn_details:
6215f3b65aba8f48b9ad4324|681073991edfa52a79549ba0|56212|1.00|ETH|ETB|SALE|http://localhost/YAGOUTPAY/transaction//Response.php|http://localhost/YAGOUTPAY/transaction//Response.php|WEB

other_details: |||||

fraud_details: |NA|

B. API Integration:

YagoutPay Test Environments for API Integration

Post URL	**https://uatcheckout.yagoutpay.com/ms-transaction-core-1-0/apiRedirection/apiIntegration**
Merchant ID	202505060003
Encryption Key	SjgTPThS1TWrD06E1FjRC3BoIgOXEh/xJ2IC+k+tVVM=
Aggregator ID	yagout

YagoutPay Production Environments

Post URL	Shared by us
Merchant ID	Shared by us
Encryption Key	Share by us
Aggregator ID	Shared by us

Encryption Logic

Encryption	AES with 256 Bits
Content-Type	Application/json
IV	0123456789abcdef
Crypt	AES-256-CBC

Request Parameters:

Merchant must send the following parameters to **YagoutPay** PG for processing an .

Section: Txn_Details (Required Parameters)

Parameter Name	Mandate	Type (length)	Description
ag_id	YES	Alphabets (11)	Aggregator id is "yagout".
me_id	YES	Numeric(20)	Merchant ID issued to you by yagout.
order_no	YES	Alphanumeric (70)	This ID is used by merchants to identify the order. Ensure that you send a **unique id** with each request. As it generates a unique payment reference number for each order which is sent by the merchant.

Amount	YES	Numeric (10,2)	This is your Transaction Amount. E.g.,"150.00"
card_no	No	Numeric (19)	Value must be blank for Aggregator MERCHANT (Non-Seamless)
country	YES	Alphabets (3)	Country code needs to be passed, C ETB for Ethiopia.
Currency	YES	Alphabets (3)	Currency in which youwant to process the transaction. ETB - Ethiopian birr
txn_type	YES	Alphabets (10)	Use Txn_Type as "**SALE**" which is static with each request
success_url	YES	Alphanumeric (200 **Characters allowed** Alphabet (A-Z), (a-z), Numbers, / (slash), _ (underscore)	This will be the return URL where response is acceptedfor successful transaction.
failure_url	YES	Alphanumeric (200) Characters allowed: Alphabet (A-Z), (a-z), Numbers, / (slash), _ (underscore)	This will be the return URL where response is accepted for failed transaction.
Channel	YES	Alphabet (10)	Use "WEB" in case of website-based payment or "MOBILE" in case of mobile app. Static value with each request.
Section: pg_details			
pg_id	No	Numeric(11)	This is the Bank Identification code assigned by yagout Team
Paymode	No	Alphabets (2)	This is the Paymode Identification code assigned by yagout Team
Scheme	No	Numeric(11)	This is the Scheme Identification code assigned by yagout Team
wallet_type	No	Alphabets	Enter the wallet e.g.: telebirr , mpesa, cbebirr, kacha
Section: card_details			

exp_month	No	Numeric (2)	Value must be blank for Aggregator Hosted(Non-Seamless)
exp_year	No	Numeric (4)	Value must be blank for Aggregator Hosted (Non-Seamless)
ship_address	Optional	Alphanumeric (400) **Characters allowed:** Alphabet (A-Z), (a-z). Numbers # (hash), Comma, circular brackets, /(slash), dot,- (hyphen) Space in between words	Customer's Shipping Address. This is required in case of physical delivery of goods.
cvv	No	Numeric (3)	Value must be blank for Aggregator Hosted(Non-Seamless)
card_name	No	Alphabets (50)	Value must be blank for Aggregator Hosted(Non-Seamless)
Section: cust_details			
cust_name	Optional	Alphabets (50)	Customer Name. This is required in case you requiredfraud protection.
email_id (**Mandatory**)	Yes	Alphanumeric (100) **Characters allowed:** Alphabet (A-Z), (a-z). Numbers @ (at), dot, _ (underscore)	Customer's Email ID. This is required in case You required fraud protection.
mobile_no (**Mandatory**)	Yes	Numeric(15)	Customer's Mobile Number. This is required in case you required fraud protection.
unique_id	No	Alphanumeric (100)	This is required in case of **Stored card feature**.
is_logged_in	Yes	Alphabet (1)	Pass "Y" if user is logged in.
Section: Bill_details			
bill_address	Optional	Alphanumeric (400) **Characters allowed:** Alphabet (A-Z), (a-z). Numbers # (hash), Comma, circular brackets, /(slash), dot,- (hyphen) Space in between words	Customer's Billing Address. This is required in case you required fraud protection.
bill_city	Optional	Alphabets (50)	Customer's Billing City. This is required in case you required fraud protection.
bill_state	Optional	Alphabets (50)	Customer's Billing State. This is required in case you required fraud protection.
bill_country	Optional	Alphabets (50)	Customer's Billing Country. This is required in case you required fraud protection.
bill_zip	Optional	Alphanumeric (20) **Characters allowed:** Alphabet (A-Z),(a-z). Numbers	Customer's Billing Zip/Pin Code. This is required in case you required fraud protection.
Section: Ship_details			

ship_city	Optional	Alphabets (50)	Customer's Shipping City. This is required in case of physical delivery ofgoods.
ship_state	Optional	Alphabets (50)	Customer's Shipping State. This is required in case of physical delivery ofgoods.
ship_country	Optional	Alphabets (50)	Customer's Shipping Country. This is required incase of physical delivery of goods.
ship_zip	Optional	Alphanumeric (20)	Customer's Shipping Zip/Pin Code. This is required in case of physical delivery of goods.
ship_days	Optional	Numeric(11)	Approximate number of days required for delivery of goods. This is required incase of physical delivery of goods.
udf_2	Optional	Alphanumeric (100) **Characters allowed:** Alphabet (A-Z), (a-z) Numbers # (hash), Comma, circular brackets, /(slash), dot, - (hyphen) Space inbetween words	This parameter can be used for sending additional information about the transaction. PG will send this parameter in the response.
address_count	Optional	Numeric (11)	If customer has more than one addresses stored in your system
Section: Item_details			
item_count	Optional	Alphanumeric (11)	Countof items purchased by customer.
item_value	Optional	Alphanumeric (200,2)	Commaseparated item wise cost
udf_3	Optional	Alphanumeric (100)	This parameter can be used for sending additional information about the transaction. PG will send this parameter in the response.
item_category	Optional	Alphanumeric (200) **Characters allowed:** Alphabet (A-Z), (a-z). Numbers # (hash), Comma, circular brackets, /(slash), dot, - (hyphen) Space in between words	Comma separated item category. E.g., "DVD"
Section: Other_details			
udf_1	Optional	Alphanumeric (100) **Characters allowed:** Alphabet (A-Z), (a-z) Numbers # (hash), Comma, circular brackets, /(slash), dot, - (hyphen) Space inbetween words	This parameter can be used for sending additional information about the transaction. PG will send this parameter in the response.

udf_4	Optional	Alphanumeric (100) **Characters allowed:** Alphabet (A-Z), (a-z). Numbers # (hash), Comma, circular brackets, /(slash), dot, -(hyphen) Space in between words	This parameter can be used for sending additional information about the transaction. PG will send this parameter in the response.
udf_5	Optional	Alphanumeric (100) **Characters allowed:** Alphabet (A-Z), (a-z). Numbers # (hash), Comma, circular brackets, /(slash), dot, -(hyphen) Space in between words	This parameter can be used for sending additional information about the transaction. PG will send this parameter in the response.
udf_6	Optional	Alphanumeric (100) **Characters allowed:** Alphabet (A-Z), (a-z). Numbers # (hash), Comma, circular brackets, /(slash), dot, -(hyphen) Space in between words	This parameter can be used for sending additional information about the transaction. PG will send this parameter in the response.
udf_7	Optional	Alphanumeric (100) **Characters allowed:** Alphabet (A-Z), (a-z). Numbers # (hash), Comma, circular brackets, /(slash), dot, -(hyphen) Space in between words	This parameter can be used for sending additional information about the transaction. PG will send this parameter in the response.

Note:

- An order details should be created for every payment.
- Order number should be unique for every transaction request.
- Make sure that youare sending the request in **JSON. Please find the format shared below:**

- **Encrypting and Decrypting** the Request and Response with the **Merchant Encryption Key**

Example of Transaction request(Plain) in JSON:


```
{
  "card_details": {
    "cardNumber": "",
    "expiryMonth": "",
    "expiryYear": "",
    "cvv": "",
    "cardName": ""
  },
  "other_details": {
    "udf1": "",
    "udf2": "",
    "udf3": "",
    "udf4": "",
    "udf5": "",
    "udf6": "",
    "udf7": ""
  },
  "ship_details": {
    "shipAddress": "",
    "shipCity": "",
    "shipState": "",
    "shipCountry": "",
    "shipZip": "",
    "shipDays": "",
    "addressCount": ""
  },
  "txn_details": {
    "agId": "yagout",
    "meId": "202505060003",
    "orderNo": "Test1",
    "amount": "1",
```

```
    "country": "ETH",
    "currency": "ETB",
    "transactionType": "SALE",
    "sucessUrl": "",
    "failureUrl": "",
    "channel": "API"
  },
  "item_details": {
    "itemCount": "",
    "itemValue": "",
    "itemCategory": ""
  },
  "cust_details": {
    "customerName": "",
    "emailId": "",
    "mobileNumber": "965680964",
    "uniqueId": "",
    "isLoggedIn": "Y"
  },
  "pg_details": {
    "pg_Id": "67ee846571e740418d688c3f",
    "paymode": "WA",
    "scheme_Id": "7",
    "wallet_type": "telebirr"
  },
  "bill_details": {
    "billAddress": "",
    "billCity": "",
    "billState": "",
    "billCountry": "",
    "billZip": ""
  }
}
```


Request after Encrypting the Plain Request:


```
lq1FilIDHAD/9Cx4innDJVF+oF2ow673w/wDf969AT48VPCrvfLwX2mdCQLYB7
7SSq5n5sOqsxT/+fizkAEMyTi8b90BLqKbVIH6aR2uYv1dk9PG0XYXF4LOp6mz
uTYinU1AydyCCNSEURPlxn//QC3YAZCt13YGi2a+ZPp8TVLE7YNQxjeqaZV9uI
oRLLl7zOXw0G5aU+xalGwSsFZLsE2GKN9hg+zQJ2VOJFZao88TBcrXhQOnMA9r
88zNlXuoqBfdQoBwvcU/ERcRevRjcYmz1lum28NRDl0//Xlnx1JBXlh94XcNl6
L5U+L7HwsAUEeVM6Fmb5+HvW7W8ZozspukVfeXdxNO2FdvcqccXrrEWpSJHop2
RtLwEqOuA3B3e2Fpbb9OBmrcCMvgRR3KgjDTll/LMuOpb/2Ad3VLLfTfwmPoyz
wTzFuKpKc3c/+A52LfNgBJ4FlmHQ37b70Nf61ORjDL4g0TdHaNb1hw7IgwOxo1
IDOpIr83kdHoeyF3MMGr11AqeEUOBpblU4NvLi6++dEUdetH0Ysglrilis8MRh
```

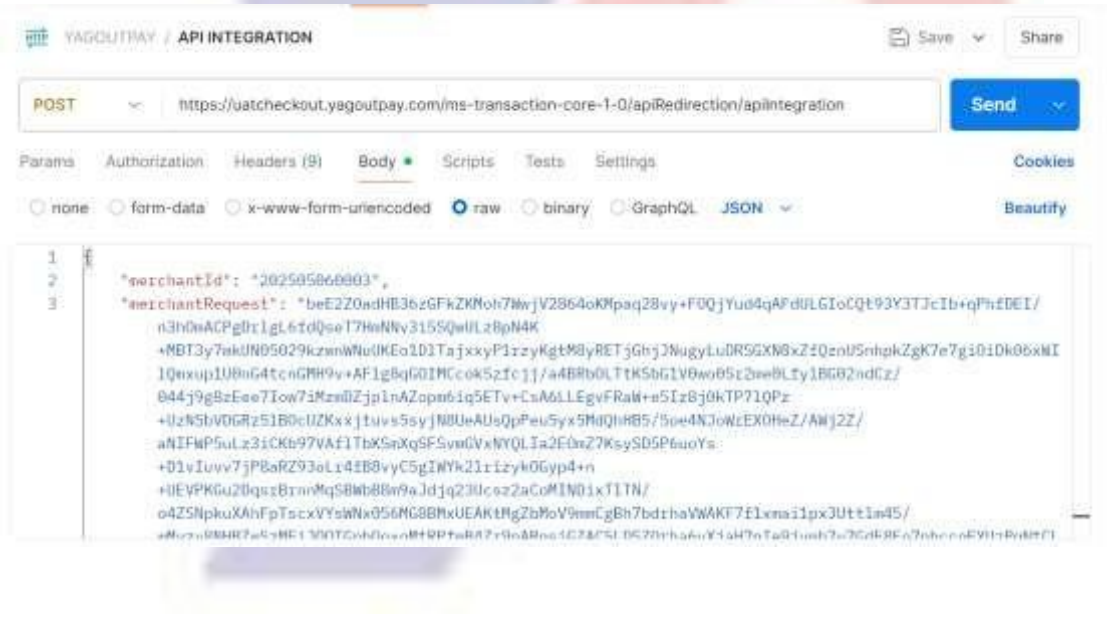
NJAhVqrlHFnxdAag9CO2nsQRDN2d+CpYD0bEej9oz0c1EOD95RS1DnZyNT1W0p
j3CaPDS9ehZlYVmOfzbDKRs7QpDJf+Lf52xU1lByCEXGy958IbsjaCo8aaBytp
sVAxC/VmkWJh1TSXeg3bZC3Ck7Lu08U6Rvfinz42qEbPQaID7RyUFQPCDwKvUK
jcspbLIruYh9dmO2gPVYU1BSPxjpqaP7BxzjfYf9y0Eb+JMMGEzqg/MaohGeeR
OruAUea6h6820bAl2/NBZ7s/N8VHdZtepwjV/tUAJTg9BnZWrbzB/VbVb9SHiP
NOWHr5SziTJV2RgmP7eXgf5EBPHnG48M2vcUPZ98Uy8Z6BuW8jv+63EunktJE5
Lo5A0rPKr4F188thbEJEyBxz1sNyD0se4K1QFLx4A6vmyqukXRrTD7EmKFIqDy
p6rKLJI/NxJhPId3VraYYtFodM47DPXbFfeDKBlHMUJRVnDCZQV64QHYMps39Q
J23ZF1KIe2aS5Jo3uqjwEc7vNC8JcnleD3QvAZ03p3TedcegBuXqVhfZM35Y5+
T3/2L+fd3MsHmjefvGblEHh27aUOHCERULTxOnXZyFMYChZOv7fFjsHY82VwQT
rMKvBED+uXjrkLFfx3BXnhzIdc24Pi9Y6rV0Jn+SNtvEz2rS+4co8U8fLgBTDL
GVXsZMaFjnbtCiIlu4eSwRHyefyNZwCtMs0cYXmsf+hqMHiyUGMeOVatO7c9dr
8n85m2bOugCGMv03KH+DiBnxFO6dK+NHgFBbWZcbFxAVAmQqzAIPlFtvSoK+G3
43zfHhaHgU8IjKgFaEnGJU1DGhOC4IQJppVCt5t3HBA/UvS/JNPRe0G8R0Qdvn
d1rMeABnXwSIsf1ngR3a7gITTvhw17W5riqHXYW/0PXzyqO8yMm8cugMtr35LA
X2+vWU7NktqKKpsGPBCLY7fuocJ7FKNesJFUVx9wWFyidJyN1A0WJ/GeG7VU0F
wTdHdTf0BykNBfCao7JO9RKIa6jWVDS8Nv75FUPtEztL1OCltUES25ZKlYAfQ6
4N6w6ug6kPgdQDEHe795fTxPRCVoha270w8D8jXjQksXgPsIWdZUhckZ5GqsIc
D73M8jpTdJg/bBKb1IlCfW2TVYYxRhBB011oq/SpXxRtVJlL3LYhpRgLB7wwAF
kUN74bJWidc+rspjiKs+CSmiI+RV6PwMh09ljMfdMzEG/lZWgPyyNBlrz7Sf8o
OmDyCFpr0CvoW9/YT8jwaK5VzGNs/SNdkClozKaL8jMZPrfLEM8rZsZqR3

Final Sample Request Format:


```
{
    "merchantId": "202505060003",
    "merchantRequest":
"lq1FilIDHAD/9Cx4innDJVF+oF2ow673w/wDf969AT48VPCrvfLwX2md
CQLYB77SSq5n5sOqsxT/+fizkAEMyTi8b90BLqKbVIH6aR2uYv1dk9PG0
XYXF4LOp6mzuTYinU1AydyCCNSEURPlxn//QC3YAZCt13YGi2a+ZPp8TV
LE7YNQxjeqaZV9uIoRLLl7zOXw0G5aU+xalGwSsFZLsE2GKN9hg+zQJ2V
OJFZao88TBcrXhQOnMA9r88zNlXuoqBfdQoBwvcU/ERcRevRjcYmz1lum
28NRDl0//Xlnx1JBXlh94XcNl6L5U+L7HwsAUEeVM6Fmb5+HvW7W8Zozs
pukVfeXdxNO2FdvcqccXrrEWpSJHop2RtLwEqOuA3B3e2Fpbb9OBmrcCM
vgRR3KgjDTll/LMuOpb/2Ad3VLLfTfwmPoyzwTzFuKpKc3c/+A52LfNgB
J4FlmHQ37b70Nf61ORjDL4g0TdHaNb1hw7IgwOxo1IDOpIr83kdHoeyF3
MMGr11AqeEUOBpblU4NvLi6++dEUdetH0Ysglrilis8MRhNJAhVqrlHFn
xdAag9CO2nsQRDN2d+CpYD0bEej9oz0c1EOD95RS1DnZyNT1W0pj3CaPD
S9ehZlYVmOfzbDKRs7QpDJf+Lf52xU1lByCEXGy958IbsjaCo8aaBytps
VAxC/VmkWJh1TSXeg3bZC3Ck7Lu08U6Rvfinz42qEbPQaID7RyUFQPCDw
KvUKjcspbLIruYh9dmO2gPVYU1BSPxjpqaP7BxzjfYf9y0Eb+JMMGEzqg
/MaohGeeROruAUea6h6820bAl2/NBZ7s/N8VHdZtepwjV/tUAJTg9BnZW
rbzB/VbVb9SHiPNOWHr5SziTJV2RgmP7eXgf5EBPHnG48M2vcUPZ98Uy8
Z6BuW8jv+63EunktJE5Lo5A0rPKr4F188thbEJEyBxz1sNyD0se4K1QFL
x4A6vmyqukXRrTD7EmKFIqDyp6rKLJI/NxJhPId3VraYYtFodM47DPXbF
feDKBlHMUJRVnDCZQV64QHYMps39QJ23ZF1KIe2aS5Jo3uqjwEc7vNC8J
cnleD3QvAZ03p3TedcegBuXqVhfZM35Y5+T3/2L+fd3MsHmjefvGblEHh
27aUOHCERULTxOnXZyFMYChZOv7fFjsHY82VwQTrMKvBED+uXjrkLFfx3
BXnhzIdc24Pi9Y6rV0Jn+SNtvEz2rS+4co8U8fLgBTDLGVXsZMaFjnbtC
iIlu4eSwRHyefyNZwCtMs0cYXmsf+hqMHiyUGMeOVatO7c9dr8n85m2bO
ugCGMv03KH+DiBnxFO6dK+NHgFBbWZcbFxAVAmQqzAIPlFtvSoK+G343z
fHhaHgU8IjKgFaEnGJU1DGhOC4IQJppVCt5t3HBA/UvS/JNPRe0G8R0Qd
vnd1rMeABnXwSIsf1ngR3a7gITTvhw17W5riqHXYW/0PXzyqO8yMm8cug
Mtr35LAX2+vWU7NktqKKpsGPBCLY7fuocJ7FKNesJFUVx9wWFyidJyN1A
```

0WJ/GeG7VU0FwTdHdTf0BykNBfCao7JO9RKIa6jWVDS8Nv75FUPtEztL1
OCltUES25ZKlYAfQ64N6w6ug6kPgdQDEHe795fTxPRCVoha270w8D8jXj
QksXgPsIWdZUhckZ5GqsIcD73M8jpTdJg/bBKb1IlCfW2TVYYxRhBB011
oq/SpXxRtVJlL3LYhpRgLB7wwAFkUN74bJWidc+rspjiKs+CSmiI+RV6P
wMh09ljMfdMzEG/lZWgPyyNBlrz7Sf8oOmDyCFpr0CvoW9/YT8jwaK5Vz
GNs/SNdkClozKaL8jMZPrfLEM8rZsZqR3"
}

Screenshot of POSTMAN FYR:



Encrypted Response Parameters:

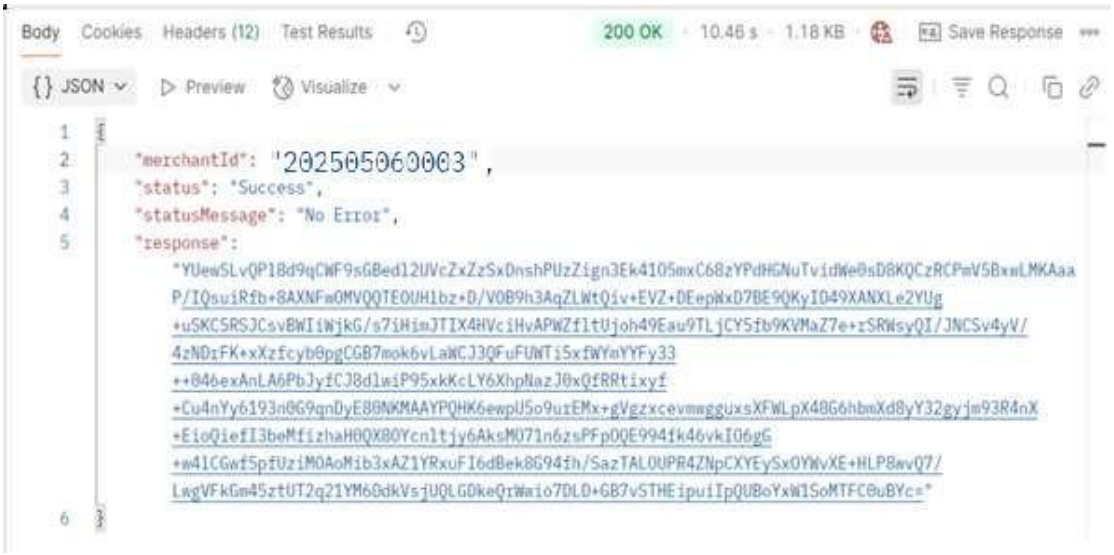
{
"merchantId": "202505060003",
"status": "Success",
"statusMessage": "No Error",
"response":
"YUewSLvQPl8d9qCWF9sGBedl2UVcZxZzSxDnshPUzZign3Ek41O5mxC6
8zYPdHGNuTvidWe0sD8KQCzRCPmV5BxwLMKAaaP/IQsuiRfb+8AXNFwOM

VQQTEOUHlbz+D/VOB9h3AqZLWtQiv+EVZ+DEepWxD7BE9QKyID49XANXL
e2YUg+uSKC5RSJCsvBWIiWjkG/s7iHimJTIX4HVciHvAPWZfltUjoh49E
au9TLjCY5fb9KVMaZ7e+rSRWsyQI/JNCSv4yV/4zNDrFK+xXzfcyb0pgC
GB7mok6vLaWCJ3QFuFUWTi5xfWYmYYFy33++046exAnLA6PbJyfCJ8dlw
iP95xkKcLY6XhpNazJ0xQfRRtixyf+Cu4nYy6193n0G9qnDyE80NKMAAY
PQHK6ewpU5o9urEMx+gVgzxcevmwgguxsXFWLpX48G6hbmXd8yY32gyjm
93R4nX+EioQiefI3beMfizhaH0QX8OYcnltjy6AksMO71n6zsPFpOQE99
4fk46vkIO6gG+w4lCGwf5pfUziMOAoMib3xAZ1YRxuFI6dBek8G94fh/S
azTALOUPR4ZNpCXYEySxOYWvXE+HLP8wvQ7/LwgVFkGm45ztUT2q21YM6
DdkVsjUQLGDkeQrWwio7DLD+GB7vSTHEipuiIpQUBoYxW1SoMTFC0uBYc
="
}

Decrypted Response:

{"txn_response":{"ag_id":"yagout","me_id":"202505060003","order_no":"0001210
0","amount":"1.00","country":"ETH","currency":"ETB","txn_date":"2025-05-09","txn
_time":"07:42:51","ag_ref":"2058701746715046364","pg_ref":"AG_20250509_704
04212c3018f132baf","status":"Successful","res_code":"0","res_message":"Successf
ul"},"pg_details":{"pg_id":"67ee846571e740418d688c3f","pg_name":"YAGOUTPAY_
MIDDLEWARE","paymode":"WA"},"fraud_details":{},"other_details":{"udf_1":"","ud
f_2":"","udf_3":"","udf_4":"","udf_5":""}}

Screenshot of POSTMAN FYR:



5. Encryption API

The Encryption API is used to encrypt the data.

Post URL	https://uatcheckout.yagoutpay.com/ms-transaction-core-1-0/othersRedirection/encryption

Method: POST
Content-Type: Application-Text

Parameters to be passed in the Header Section:
me_id : 202505090001(This is just a Test MID)

Plain Request:

{

"me_id": "202505090001",

"amount": "100"

}

///// (Enter the data that you want to encrypt)

Response:

{"Status":"Suceess","Response":"SG1a1jGJ84L16p9JARDmmSHZdffP++0FzQ1Ppa7Kt4qZkl23vPX3DYEXAgQmP2u7ZrYvYzGZA1muQnnVCMWYCw=="}

The above highlighted response is the encrypted data.

6. Decryption API

The Decryption API is used to decrypt the data.

Post URL	https://uatcheckout.yagoutpay.com/ms-transaction-core-1-0/othersRedirection/decryption

Method: POST
Content-Type: Application-Text

Parameters to be passed in the Header Section:
me_id : 202505090001(This is just a Test MID)

Plain Request:

SG1a1jGJ84L16p9JARDmmSHZdffP++0FzQ1Ppa7Kt4qZkl23vPX3DYEXAgQmP2u7ZrYvYzGZA1muQnnVCMWYCw==

///// (Enter the data that you want to decrypt)

Response:

{Status=Suceess, Response= {

"me_id": "202505090001",

"amount": "100"

} }

The above highlighted response is the decrypted data.

...THANK YOU